

Concept Map Report: OSI Data Units, Addressing, and Error Control

Networking Fundamentals — OSI Model Analysis

Althaf A
(192521257)

Question 2: Concept Map of Data Units Across OSI Layers

Task: Construct a concept map linking data units (Segment, Packet, Frame, Bits) with their respective OSI layers, and analyze how the transformation of data units supports transmission.

2.1 What is a 'Data Unit'?

Every message you send over a network — an email, a photo, a chat message — starts as raw information. As this information travels down through the layers of the OSI (Open Systems Interconnection) model on the sending computer, each layer wraps it with its own control information (called a header, and sometimes a trailer). Each time this wrapping happens, the data is given a new name. This renamed, re-wrapped unit of data is called a Protocol Data Unit (PDU) — simply, 'the data unit at a particular layer.'

So a 'data unit' is just the name given to the message at a specific stage of its journey through the layers — the same underlying information, but with a different 'outfit' at each layer.

2.2 The Concept Map: Layer to Data Unit

OSI Layer	Data Unit Name	What Happens at This Layer
Application / Presentation / Session (Layers 7,6,5)	Data	User information is created and formatted (e.g., an HTTP request, an email body).
Transport (Layer 4)	Segment	Data is broken into pieces; port numbers, sequence numbers, and error-checking info are added.
Network (Layer 3)	Packet	Source and destination IP addresses are added so the data can be routed across networks.
Data Link (Layer 2)	Frame	MAC addresses and a trailer (for error checking) are added so the data can travel across one physical link.
Physical (Layer 1)	Bits	The frame is converted into raw electrical, optical, or radio signals (1s and 0s) for transmission.

Concept Map (visual form)

```
graph TD
    A["Data (Application / Presentation / Session)"] --> B["| Transport Layer adds Header --> SEGMENT"]
    B --> C["v"]
    C --> D["Segment"]
    D --> E["| Network Layer adds IP Header --> PACKET"]
    E --> F["v"]
    F --> G["Packet"]
    G --> H["| Data Link Layer adds MAC Header + Trailer --> FRAME"]
    H --> I["v"]
    I --> J["Frame"]
    J --> K["| Physical Layer converts to signals --> BITS"]
    K --> L["v"]
```

Bits ----> [transmitted over cable / air / fiber] ----> Receiver reverses the process

Summary: Each OSI layer produces its own named data unit — Data, Segment, Packet, Frame,

Bits, from top to

bottom — because each layer adds specific control information needed for that stage of the

journey.

The process of wrapping data with a new header (and sometimes trailer) at each layer, as it moves down the stack, called ENCAPSULATION. The reverse process at the receiver — removing each header layer by layer as data moves up the stack — is called DECAPSULATION. This transformation is what makes reliable, organized transmission possible. Here is why:

Layer independence: Because each layer only adds and reads its own header, layers can be designed, upgraded, or replaced independently. For example, you can switch from Wi-Fi to Ethernet (Physical/Data Link layers) without changing how the Network or Transport layers work.

Addressing at the right scope: A Segment doesn't know about IP addresses, and a Packet doesn't know about MAC addresses. By adding the right addressing information at the right layer (port numbers at Transport, IP addresses at Network, MAC addresses at Data Link) each layer solves exactly the problem it needs to solve — no more, no less.

Error and flow control at the right scope: Frames carry a trailer used for detecting transmission errors on that one physical link; Segments carry sequence numbers used for ensuring correct order and retransmission end-to-end. Splitting these jobs across layers keeps each one simple and efficient.

Reassembly is possible: Because each layer's header records information needed to undo its own transformation (e.g., sequence numbers in a Segment header), the receiving side can decapsulate step by step and reconstruct the original Data exactly, even though it may have been broken into many Packets or Frames along the way.

Real-world examples

Example 1 — Streaming a video call: The video/audio Data is broken into Segments (Transport) so it can be sent as many small chunks; each Segment becomes a Packet (Network) so it can be routed across the internet to the other participant; each Packet becomes one or more Frames (Data Link) to cross each Wi-Fi or Ethernet hop; finally, Bits carry it as electrical or radio signals.

Example 2 — Loading a webpage: Your browser's HTTP request is Data; Transport (TCP) turns it into Segments with sequence numbers so the web server can reassemble the request correctly even if packets arrive out of order; Network (IP) wraps each Segment into a Packet addressed to the server's IP; Data Link wraps each Packet into a Frame addressed to the next device's MAC address (e.g., your home router).

Example 3 — Sending a large file over a company LAN: The file (Data) is too large for one Frame, so it is split into many Segments, then Packets, then Frames — each Frame is small enough to fit the network's maximum size limit (MTU), so transmission stays efficient and errors affect only a small piece of the file rather than the whole thing.

Summary: Encapsulation transforms one large piece of Data into progressively smaller, addressed, and checkable units (Segment, Packet, Frame, Bits). This division of responsibility across layers is what allows large, complex

messages to be transmitted reliably over shared, imperfect physical media, and reassembled

Question 3: MAC Address vs IP Address — Local and Global Delivery

correctly to the

destination.

Given concept map: Data Link Layer to MAC Address to Local Delivery, and Network Layer to IP Address to Global Delivery.

Data Link Layer --> MAC Address --> Local Delivery

Network Layer --> IP Address --> Global Delivery

Task: Explain how these relationships ensure complete data delivery across networks.

3.1 Two Different Addressing Jobs

Delivering data across the internet is really two delivery problems solved together:

Problem A — Getting the data to the right network, anywhere in the world (global delivery). This is solved by the Network Layer using the IP (Internet Protocol) address — a logical address that identifies which network and which host, no matter where that host physically is.

Problem B — Getting the data across one physical link, from one device to the very next device (local delivery). This is solved by the Data Link Layer using the MAC (Media Access Control) address — a physical address unique on that local network segment (e.g., your home LAN or office LAN).

3.2 Why Both Are Needed — Hop by Hop Delivery

A packet travelling from your laptop to a server on the other side of the world passes through many routers. The source and destination IP addresses never change during the whole trip — this is what makes global delivery possible: every router along the way can look at the destination IP and decide which direction to forward the packet all the way to the correct network.

However, IP addresses alone cannot move a packet across a physical wire or radio link — that job belongs to MAC addresses. At every single hop (laptop-to-router, router-to-router, router-to-destination), the Data Link layer wraps the packet in a new Frame with a new source and destination MAC address — the MAC addresses of the two devices on that link only. Once the frame arrives, the receiving device strips off that MAC header, checks the IP address, and if it is not the final destination, sends it out again in a brand-new frame with a new pair of MAC addresses for the next hop.

Concept Map in Action

Step	What Stays the Same	What Changes at Each Hop
Laptop to Home Router	Source IP, Destination IP	MAC of laptop to MAC of home router
Home Router to ISP Router	Source IP, Destination IP	MAC of home router to MAC of ISP router
...multiple router hops...	Source IP, Destination IP	New MAC pair at every hop
Last Router to Destination Server	Source IP, Destination IP	MAC of last router to MAC of destination server

3.3 How ARP Connects the Two

A device usually only knows the destination's IP address, not its MAC address. To find the MAC address of the next hop, devices use ARP (Address Resolution Protocol), which asks 'who has this IP address?' on the local network and receives back the matching MAC address. This is the bridge that lets the Network layer's global addressing (IP) work together with the Data Link layer's local addressing (MAC).

Real-world examples

Example 1 — Sending a message abroad: The destination IP address (identifying the recipient's network) stays the same across the entire journey through many international routers, while the MAC addresses change at every single router hop the message physically passes through.

Example 2 — Office network to a cloud server: Your computer's frame first uses the MAC address of your office router (local delivery, one hop). The router then re-frames the packet with a new MAC address to reach the ISP, and so on, while the IP address of the cloud server never changes (global delivery, whole journey).

Example 3 — Home Wi-Fi printing: You print from your phone to a printer on the same Wi-Fi network. The phone can directly discover the printer's MAC address via ARP and deliver frames locally in a single hop — global (IP) routing through the internet is not even needed.

Summary: MAC addresses handle LOCAL delivery — moving a frame across exactly one physical link — and change at every hop. IP addresses handle GLOBAL delivery — identifying the true source and destination anywhere in the world — and stay constant for the whole journey. Together, hop-by-hop local delivery (MAC) chained end-to-end under a constant global address (IP) is what allows a packet to travel across many different networks and still arrive at exactly the right destination.

Question 4: Error Control Across Data Link and Transport Layers

Given concept map: Error Control to Data Link Layer to Error Detection, and Error Control to Transport Layer to Error Correction.

Error Control --> Data Link Layer --> Error Detection

Error Control --> Transport Layer --> Error Correction

Task: Analyze how the interaction between these layers improves reliability in data transmission.

4.1 Why Errors Happen

Physical transmission media (copper cable, fiber, radio waves) are never perfect. Electrical noise, interference, signal weakening (attenuation), or collisions can flip bits from 0 to 1 or 1 to 0 while data travels. If nothing checked for this, corrupted data could silently reach applications and cause failures — so different OSI layers each play a role in catching and fixing these errors.

4.2 Data Link Layer: Error Detection

The Data Link layer deals with a single physical hop at a time — for example, one Wi-Fi link or one Ethernet cable. Because it only manages one link, it is well-placed to quickly check whether a Frame arrived undamaged. It typically adds a trailer to each Frame containing a value calculated from the frame's contents, such as a CRC (Cyclic Redundancy Check) or checksum.

The sender calculates the CRC/checksum from the Frame's data and attaches it.

The receiver recalculates the same value from the Frame it received and compares it to the attached value.

If the two values do not match, the Frame is considered corrupted and is discarded — the Data Link layer usually detects the problem but does not fix it itself (some protocols on unreliable links, like Wi-Fi, do request a local retransmission, but many, like Ethernet, do not).

4.3 Transport Layer: Error Correction

The Transport layer (e.g., TCP) is responsible for the entire end-to-end journey from the original sender to the final receiver, no matter how many hops, routers, or links are involved. Because it sees the full picture, it can guarantee that the complete, correct data eventually arrives, not just that one hop was error-free.

Every Segment is given a sequence number, so the receiver can tell if any Segments are missing, duplicated, or arrived out of order.

The receiver sends back acknowledgements (ACKs) confirming which Segments arrived correctly.

If the sender does not receive an ACK in time (implying the Segment was lost or discarded somewhere), it retransmits that Segment. This process is called ARQ (Automatic Repeat reQuest), and it is how the Transport layer 'corrects' the error — not by fixing bad bits directly, but by getting a fresh, correct copy resent until the data is complete.

4.4 How the Two Layers Work Together

These two layers form two lines of defense, each suited to what it can see:

Aspect	Data Link Layer	Transport Layer
Scope	One physical link (one hop)	Entire end-to-end path (all hops)

Mechanism	CRC / checksum in frame trailer	Sequence numbers + acknowledgements (ARQ)
Action on error	Detects and discards the bad frame	Detects the missing/bad segment and retransmits it
Result	Prevents locally corrupted frames from moving further up the stack	Guarantees complete, correctly ordered data arrives at the final destination

Because the Data Link layer discards clearly corrupted frames close to where the damage happened, it prevents wasted effort — there is no point sending obviously broken data further across the network. But because a Data Link layer can only see its own single hop, it cannot know if a Packet was dropped somewhere else entirely, or if congestion on a distant router caused loss. That is exactly the gap the Transport layer fills: by tracking the full journey with sequence numbers and acknowledgements, it notices any missing piece, regardless of which hop lost and requests a fresh copy. Together, this layered interaction means: local damage is caught and removed quickly (efficient), while end-to-end completeness is still guaranteed (reliable) — the system does not depend on any single hop being perfect, only on the overall retransmission mechanism catching whatever slips through.

Real-world examples

Example 1 — Downloading a software update: If Wi-Fi interference corrupts a frame midway through downloading, the Data Link layer on that hop detects the CRC mismatch and drops the frame; TCP at the Transport layer eventually notices the corresponding segment was never acknowledged and resends it, so the final downloaded file is still complete and correct.

Example 2 — Video call over a shaky mobile network: Data Link error detection quietly discards corrupted frames on the mobile radio link as they happen, hop by hop, while the Transport layer manages retransmission, keeping the overall call usable.

Example 3 — Online banking transaction: A single corrupted frame on a router hop is dropped immediately (Data Link detection), and TCP's acknowledgement/retransmission mechanism ensures the exact banking data segment is resent and received intact, which is critical since banking data cannot be allowed to arrive incomplete or altered.

Summary: The Data Link layer provides fast, local error DETECTION on each individual hop (catch and discard), while the Transport layer provides end-to-end error CORRECTION across the whole path (detect what's missing and retransmit it). This two-layer interaction — local filtering plus a global guarantee — is what makes overall data transmission reliable despite imperfect physical links.

Quick Revision

Data units by layer: Data (upper layers) to Segment (Transport) to Packet (Network) to Frame (Data Link) to Bits (Physical).

Encapsulation adds a header (and sometimes trailer) at each layer going down; decapsulation removes them going back up at the receiver.

MAC address = local, physical, changes every hop. IP address = global, logical, stays the same the whole journey.

ARP maps a known IP address to the correct MAC address for the next hop.

Data Link layer = error DETECTION per hop (CRC/checksum, discard bad frames).

Transport layer = error CORRECTION end-to-end (sequence numbers, ACKs, retransmission/ARQ).

Questions to Check Understanding

1. Why does the same message get a different name (Data, Segment, Packet, Frame, Bits) at each OSI layer?
2. If a packet crosses 5 routers to reach its destination, how many times does its MAC address change, and how many times does its IP address change?

3. Why can the Data Link layer only detect an error and not always correct it on its own?